TEK-UP
Private Higher School
of Technologies & Engineering

TEK-UP
University

Framework django

Support de cours/TD Framework django

Classes: CII-3-GLSI

Aperçu du contenu (1/2)

- Framework django: Big Picture
- Framework django: Architecture MVT
- Installation des outils, premier projet django
- Anatomie, concepts Projet/Application django
- Application web "Meeting Planner": Maitriser l'architecture, découvrir les concepts de base
- Couche Modèle:
  - Concept ORM
  - Modèle: définition, manipulation, relationships
  - Configuration de la base de données
- URL Mapping et views
- Templates
- Django Forms

Aperçu du contenu (2/2)

- Interface Admin: découverte, publier, manipuler des modèles, personnaliser
- CBV (Class Based Views)
- Cookies, Sessions et Middlewares avec Django
- Sécurité: Authentification, permissions, groupes.
- REST API: Django REST Framework
- Django Signals
- Channels (web sockets)

Évaluation

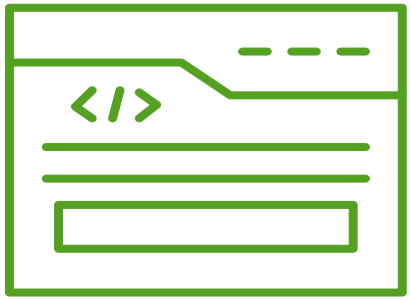
- Mini-projet: 40%
- Examen: 60%

Objectifs

- Maitriser l'architecture du framework **django**
- Développer une application web avec le framework **django**

Prérequis





Développement web
HTML, CSS, JavaScript



Base de données
Concepts de base
Table, rows
Insert, update



Framework web Python

Développer rapidement des applications web

Avec le minimum de code

Développé entre 2003 et 2005

Django: backend Framework

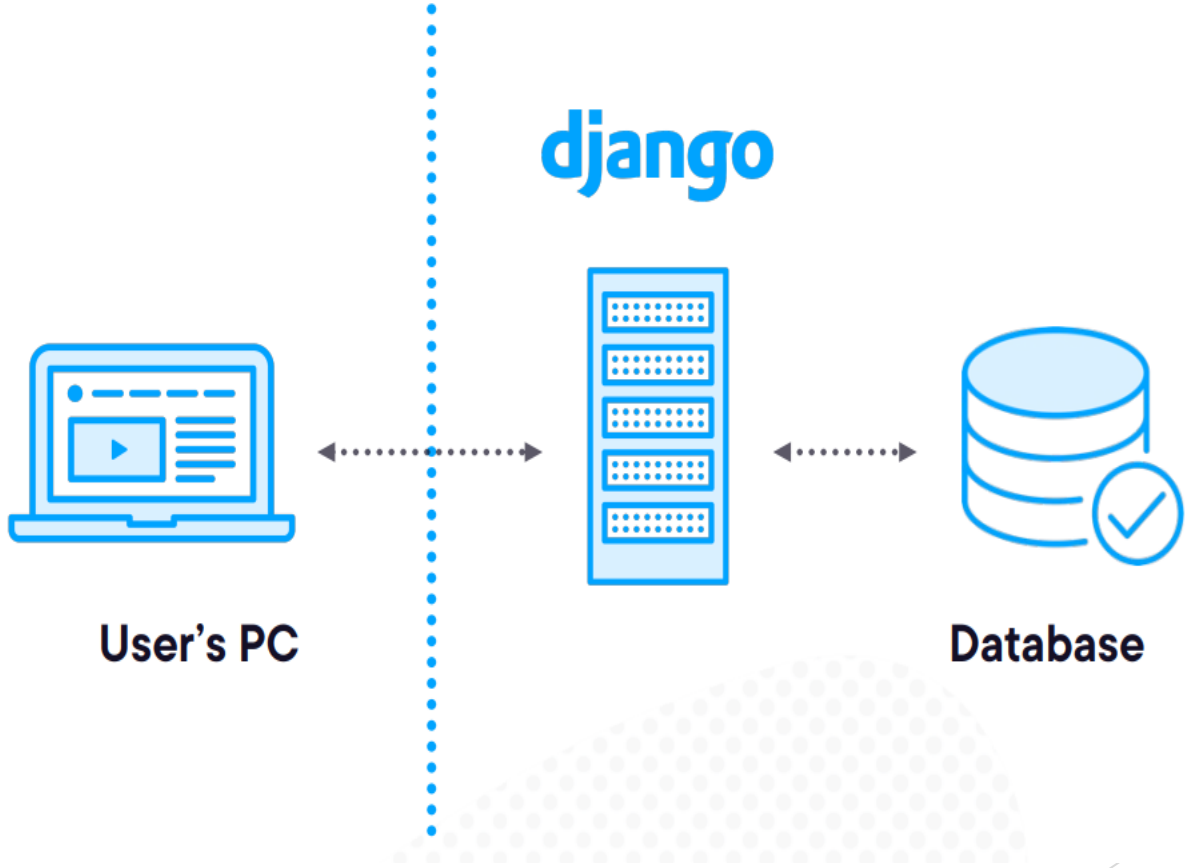
Django runs on backend

Server side

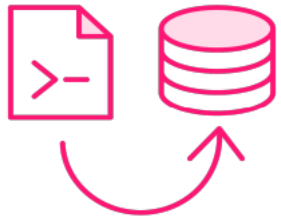
Read/write to DB

Render HTML pages

Combine with React, Angular etc.



Django features



Data model Layer



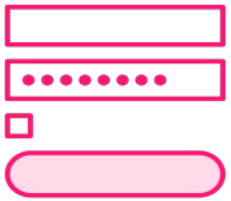
Admin interface



Template language



URL mapping

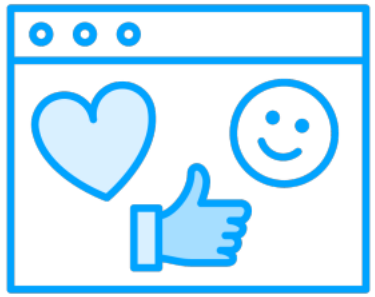


Forms



Packages

Use cases



Social networks



News sites

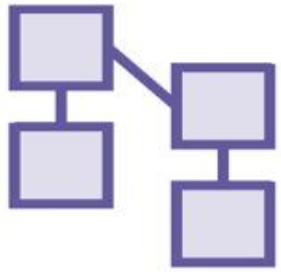


E-commerce



Serving API and backends

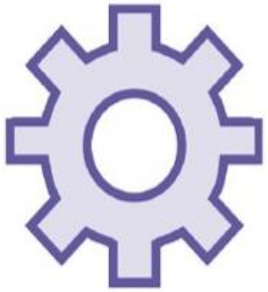
django: Architecture MVT



Model

Représente les données,
Assure le mapping
Objet/Relationnel

MVC: "Controller"



View

Reçoie une requête http,
effectue un traitement,
retourne une réponse http

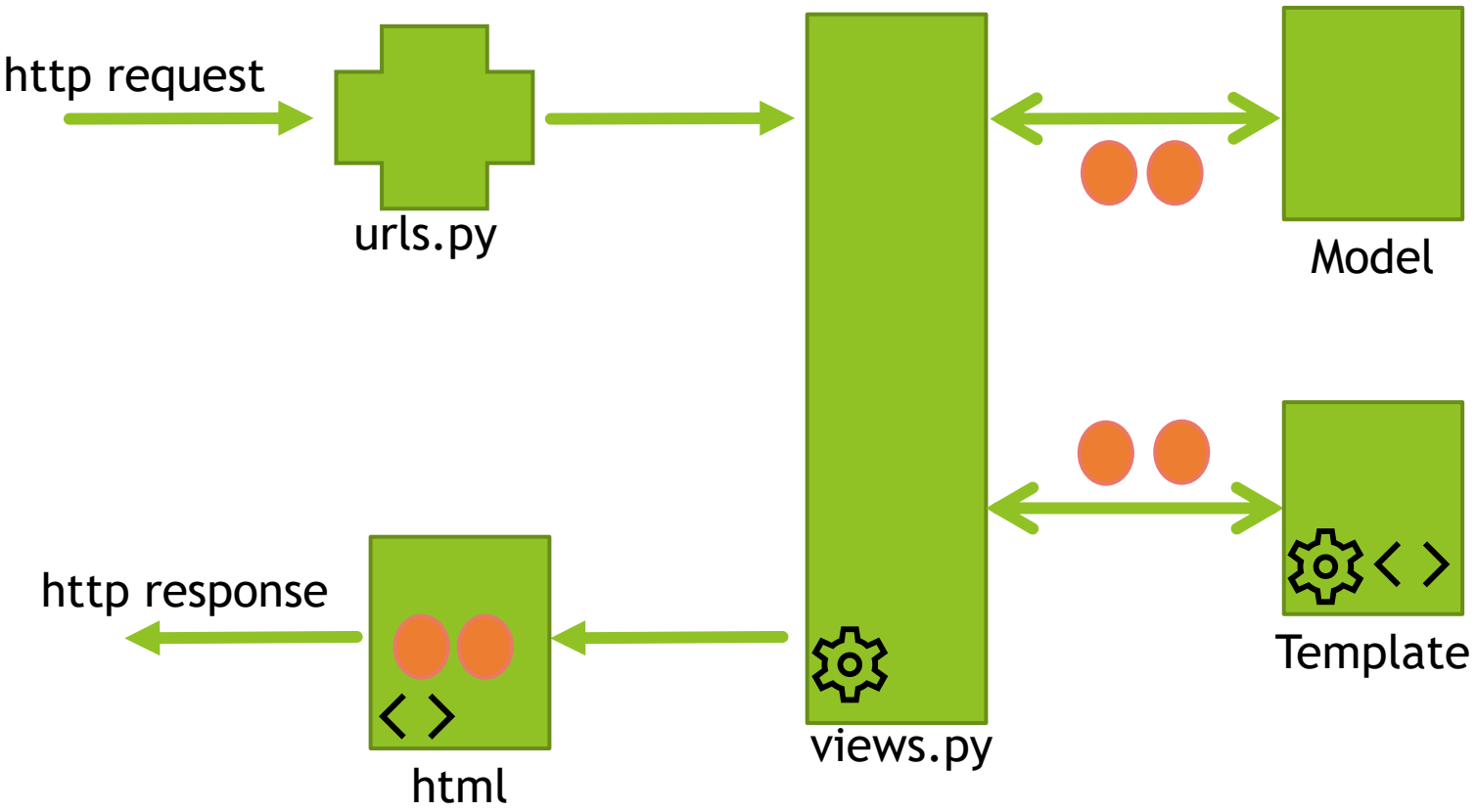
MVC: "View"



Template

Définit, génère la
présentation HTML

django: Architecture MVT







Système d'exploitation: windows, Linux, Mac OS



Python: Latest version 3.12



Éditeur de code: VS code, PyCharm

django: Première application

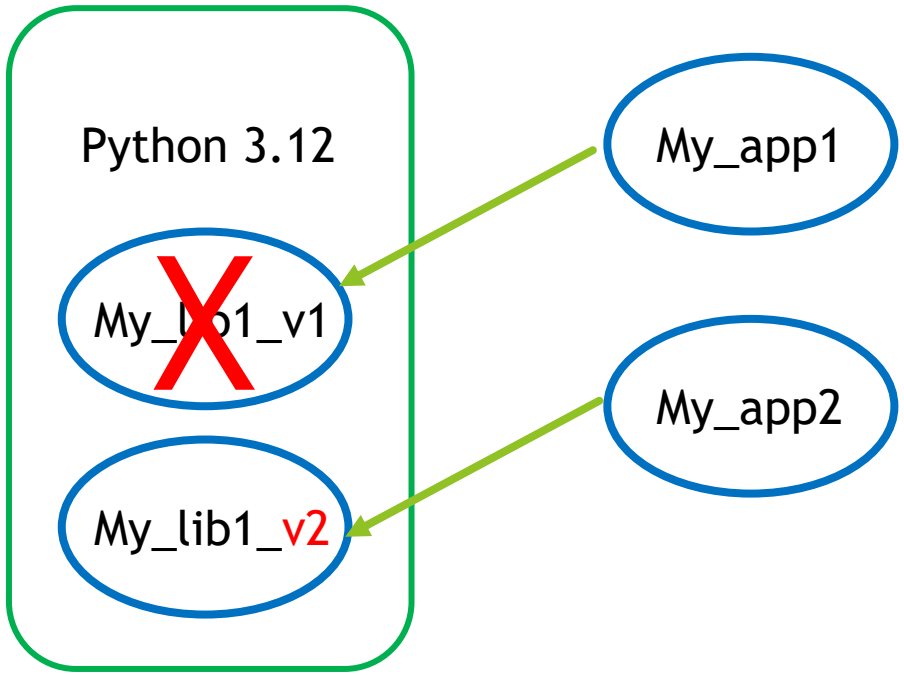
Étape par Étape

- Installation et configuration des outils
- Création et activation d'un environnement virtuel pour le projet
- Installation du framework django (virtual env activé)
- Sauvegarder les dépendances
- Création du projet django nommé **hello_word_project**
- Création de l'application **pages**
- Déclaration de l'application **pages**
- Première page web

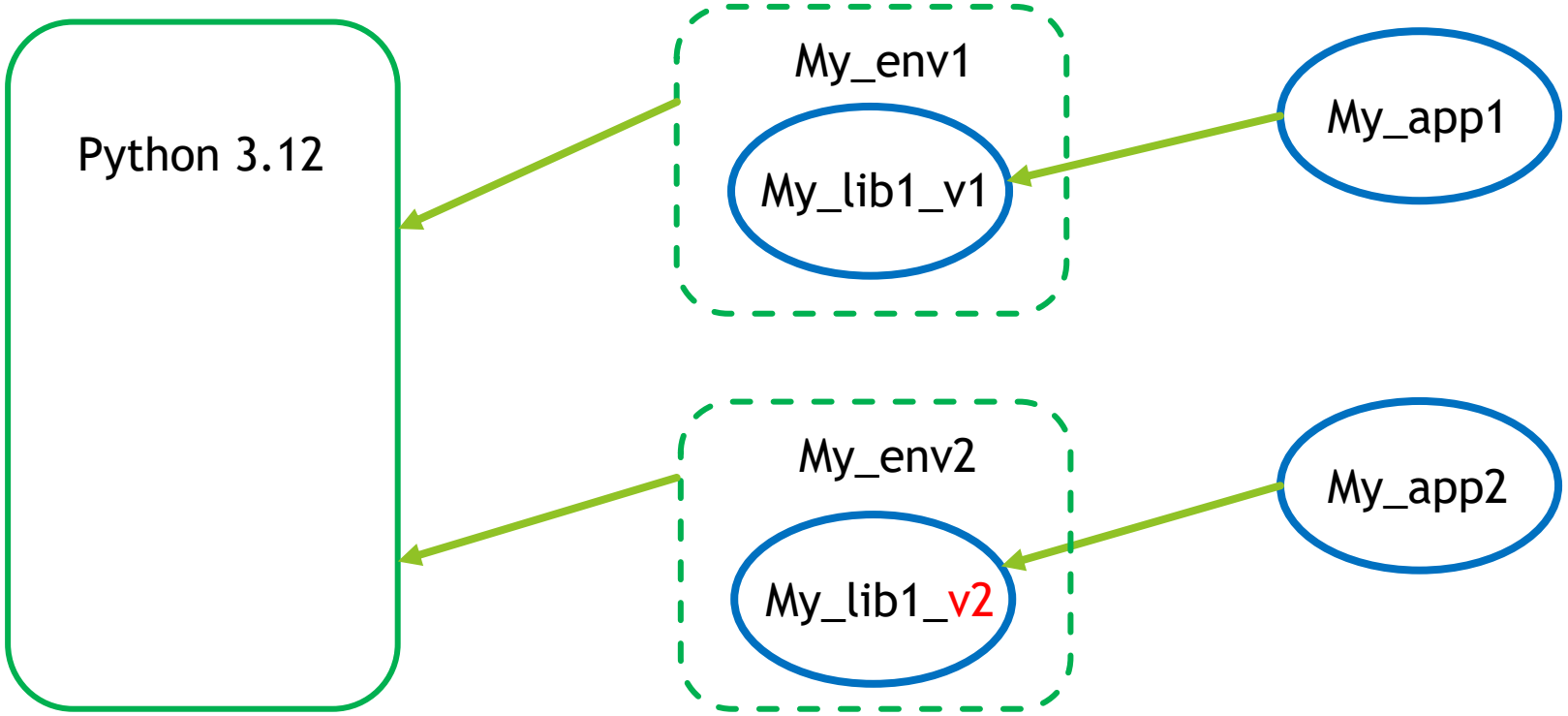
Installation et configuration des outils

- Installer Python: https://www.python.org/downloads/
- Installer VS Code ou PyCharm

Environnement virtuel: problématique



Environnement virtuel: solution



Environnement virtuel

Étape par Étape



- Ne pas installer les packages python globalement
- Toujours travailler dans un environnement virtuel
- Éviter les conflits de dépendances
- Travailler dans un contexte isolé.

Environnement virtuel

Étape par Étape

#créer un dossier pour le projet

$ cd framework_django\projects

$ mkdir helloword

$ cd helloword

#créer et activer l'environnement virtuel

$ python -m venv my_env

$ my_env\Scripts\activate

(my_env) $ python -m pip install django

$ source my_env/bin/activate

Sauvegarder les dépendances

Étape par Étape

```
# sauvegarder les dépendances
(my_env) $ pip freeze > requirements.txt
```

requirements.txt

```
asgiref==3.8.1
Django==5.1.1
sqlparse==0.5.1
tzdata==2024.1
```

```
# insaller les dépendances
(my_env) $ python -m pip install -r requirements.txt
```

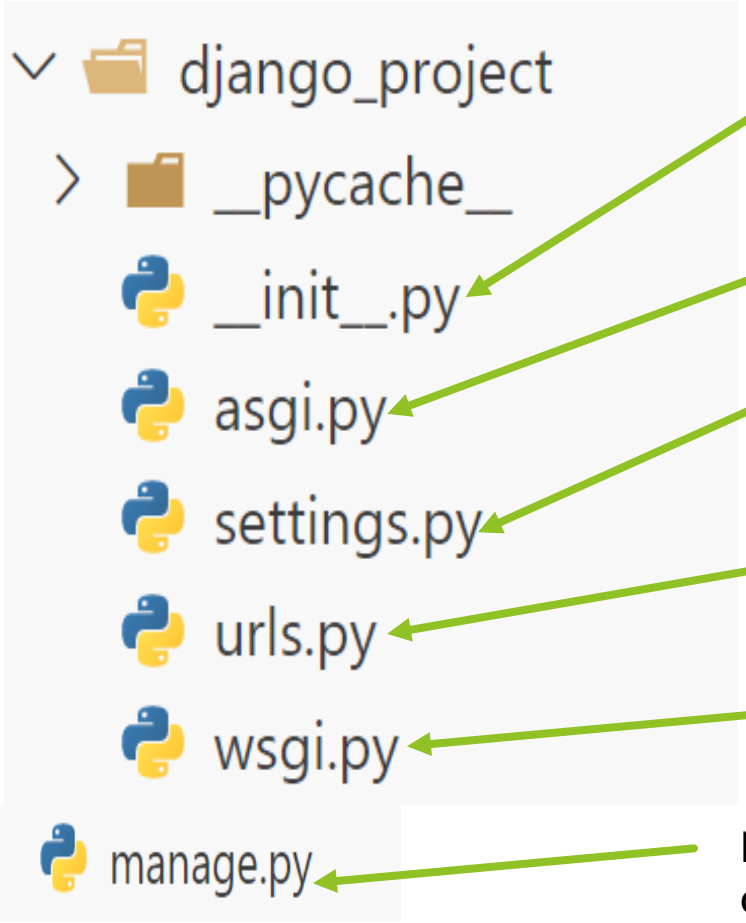
Créer le projet django_project

Étape par Étape

```
# créer le projet django_project
(my_env) $ django-admin startproject django_project  .
# lancer le projet dans VS code
(my_env) $ code .
```

django_project: Anatomie

Étape par Étape



Marque le répertoire comme package => pouvoir importer les composants

Standard Python pour application et serveur asynchrone

Contrôle l'application django via des configurations

Binding entre requête http et traitement associé

Web Server Gateway Interface

Ne fait pas partie du projet, mais utile pour exécuter des commandes: runserver, créer nouvelle application,…

Lancer le projet

Étape par Étape

```
# démarrer le projet

(my_env) $ python manage.py runserver
```

```
# appliquer les migrations

(my_env) $ python manage.py migrate
```

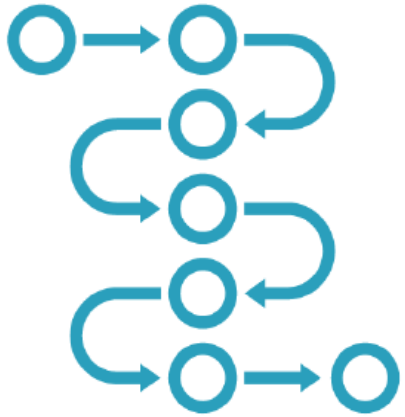
Migrations

Étape par Étape



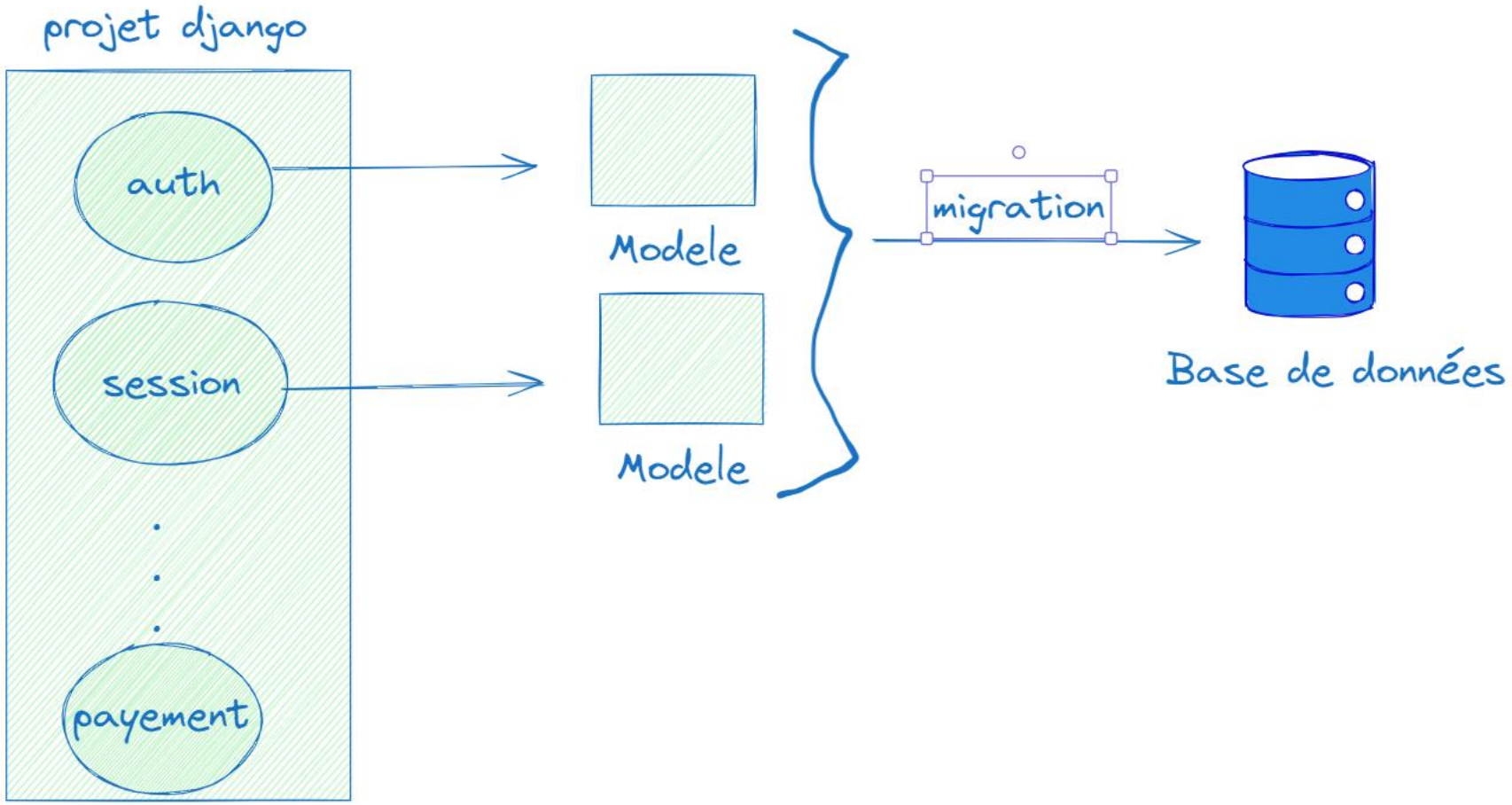
Models

Classes Python
Associées aux tables de
la BD

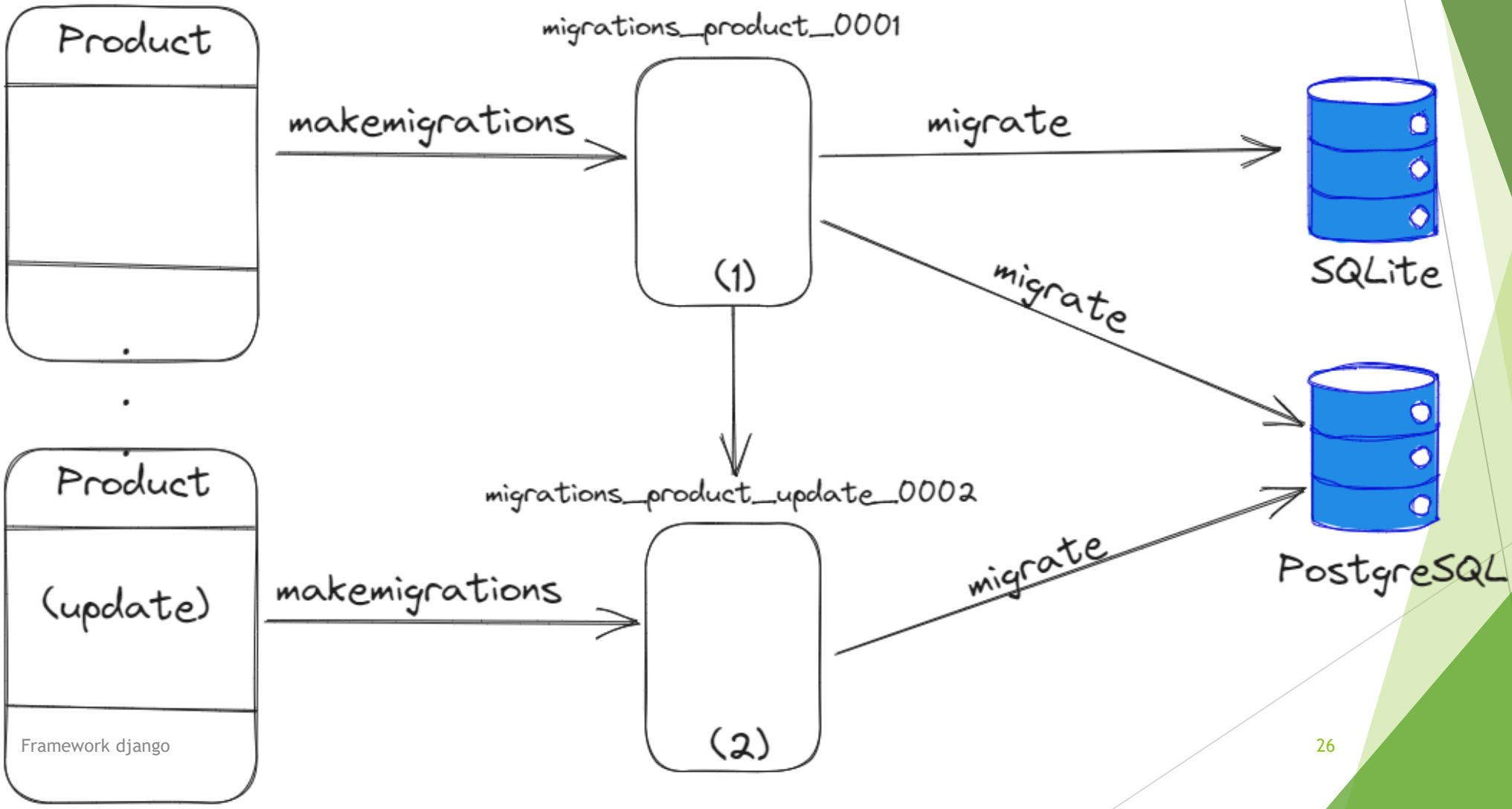


Migrations

Ensemble de scripts
décrivant le schéma de
la BD


projet django
auth
session
payement
Modele
Modele
migration
Base de données

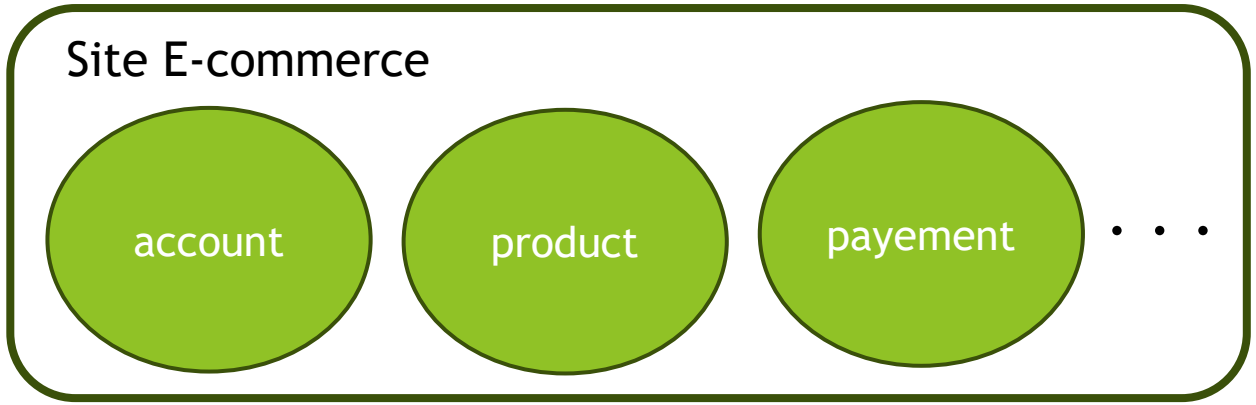
Models et Migrations



Application django: pages

Étape par Étape

- django utilise les concepts **projet** et **application** pour maintenir le code "clean"
- un projet django peut contenir plusieurs applications
- Chaque application contrôle une fonctionnalité isolée du projet.



Application django: pages

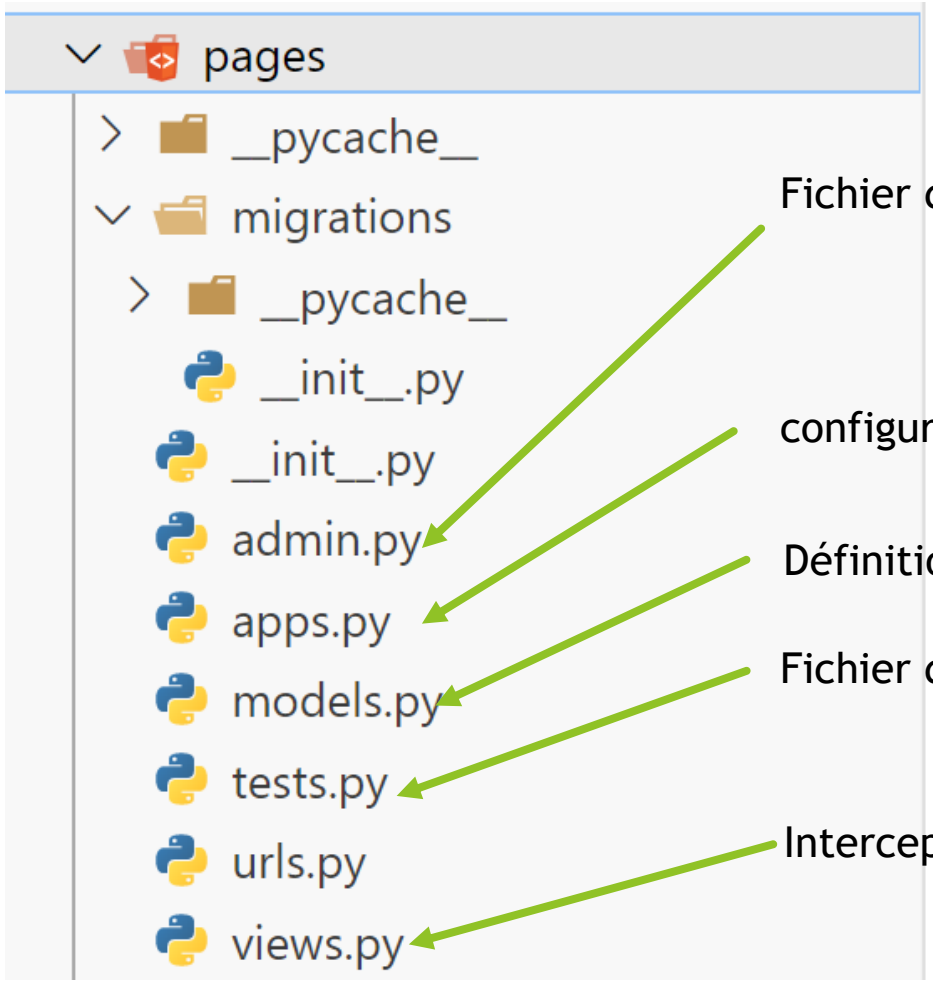
Étape par Étape

```
# créer l'application pages

(my_env) $ python manage.py startapp pages
```

Application django: Anatomie

Étape par Étape



Fichier de configuration pour le module Admin-app

configuration de l'application pages

Définition du modèle

Fichier dans lequel on code les tests

Intercepter les requêtes, traitement, retourner une réponse

Déclaration de l'application pages

Étape par Étape

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    "pages",#new
]
```

Première page web: views.py

Étape par Étape

views.py U

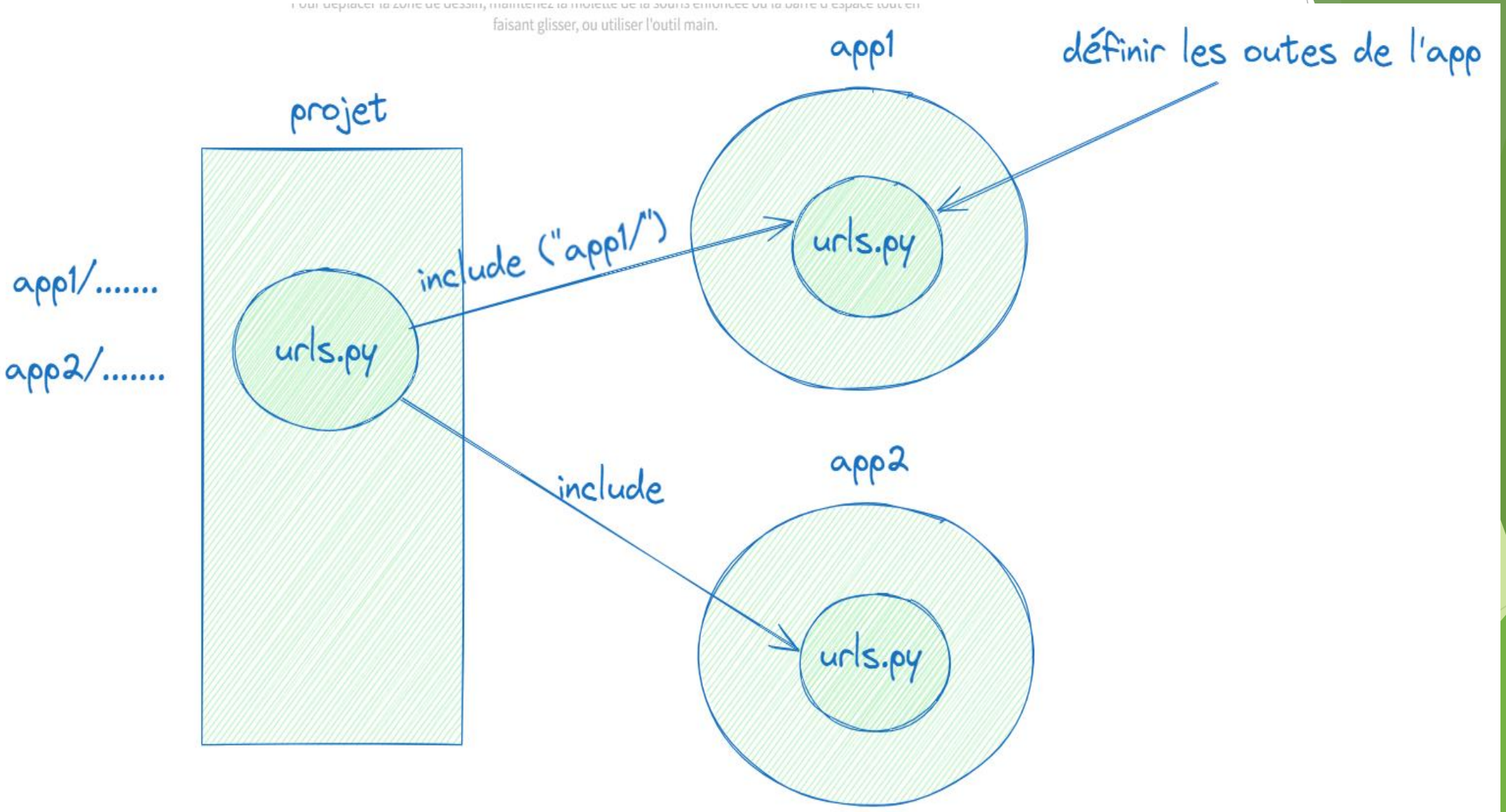
pages > views.py > ...

```python
from django.shortcuts import render
from django.http import HttpResponse

# Create your views here.

def home_page_view(request):
    return HttpResponse("Hello word")

```


app1
définir les outes de l'app
projet
app1/.....
app2/.....
urls.py
include ("app1/")
urls.py
include
app2
urls.py

Première page web: pages.urls.py

Étape par Étape

urls.py U

pages > urls.py > ...

```python
from django.urls import path

from .views import home_page_view

#domain.com/pages/.....
urlpatterns=[
    path('home/',home_page_view, name="home"),
]
```

Première page web: urls.py

Étape par Étape

urls.py U

django_project > urls.py > ...

```
17  from django.contrib import admin
18  from django.urls import path,include
19
20  urlpatterns = [
21      path('admin/', admin.site.urls),
22      path('pages/',include("pages.urls")),
23  ]
```



- Django: framework web Python
- Développement web simple et rapide
- Piles incluses: ORM, Admin module, ...
- Architecture MVT (Model, View, Template)
- Un projet django, plusieurs applications
- Virtual environnement

Application: Meeting Planner

Résultats

- Maitriser l'architecture MVT
- Découvrir les concepts de base de django:
  - Model
  - View
  - Template
  - URLs
  - Admin panel
  - Forms

Meeting Planner

- Permet de planifier les réunions
- Planifier une **réunion:**
  - Définir la date, la durée
  - affecter une **salle** à cette réunion

Meetings-planner: Code source



Welcome to the Meeting Planner!

New Meeting

Meetings

- Seance 01
- Seance 02
- Seance 03
- Seance 04

Démo de l’application

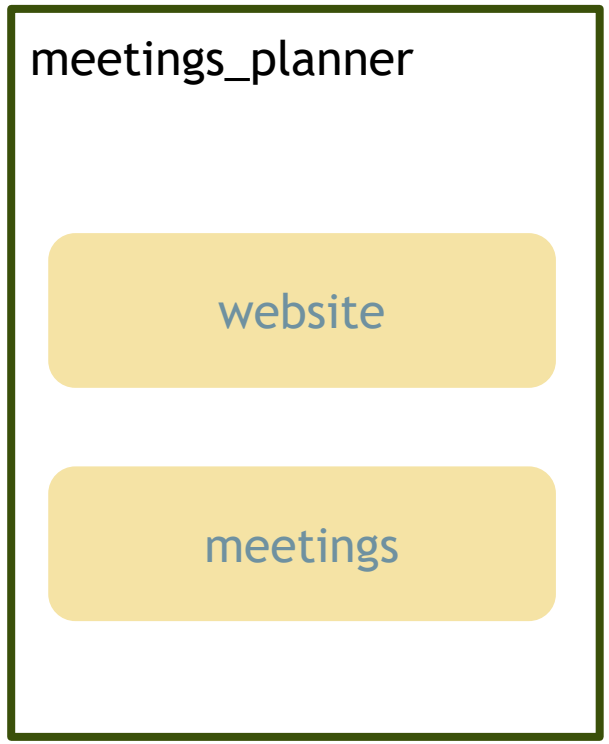
Démarche

Étape par Étape

- Créer le projet **meetings_planner**
  - Créer et activer l'environnement virtuel
  - Installer les dépendances
  - Créer le projet et les applications django
- Implémenter les pages web (accueil, about)
- Implémenter le modèle **Meeting**
- Admin Panel:
  - Créer un **superuser**
  - Gérer le modèle **Meeting**
- Gérer les **Meeting**
- implémenter le modèle Room
- Définir l'association Meeting-----Room

Architecture du projet

Étape par Étape



django apps



Python package

Contient models, views, templates, urls

Les projets django contiennent plusieurs apps

Les Apps peuvent être réutilisées

Maintenir les apps petites et simples

Création du projet

Étape par Étape

```
#créer un dossier pour le projet
$ cd framework_django\projects
$ mkdir meetings_planner
$ cd meetings_planner
```

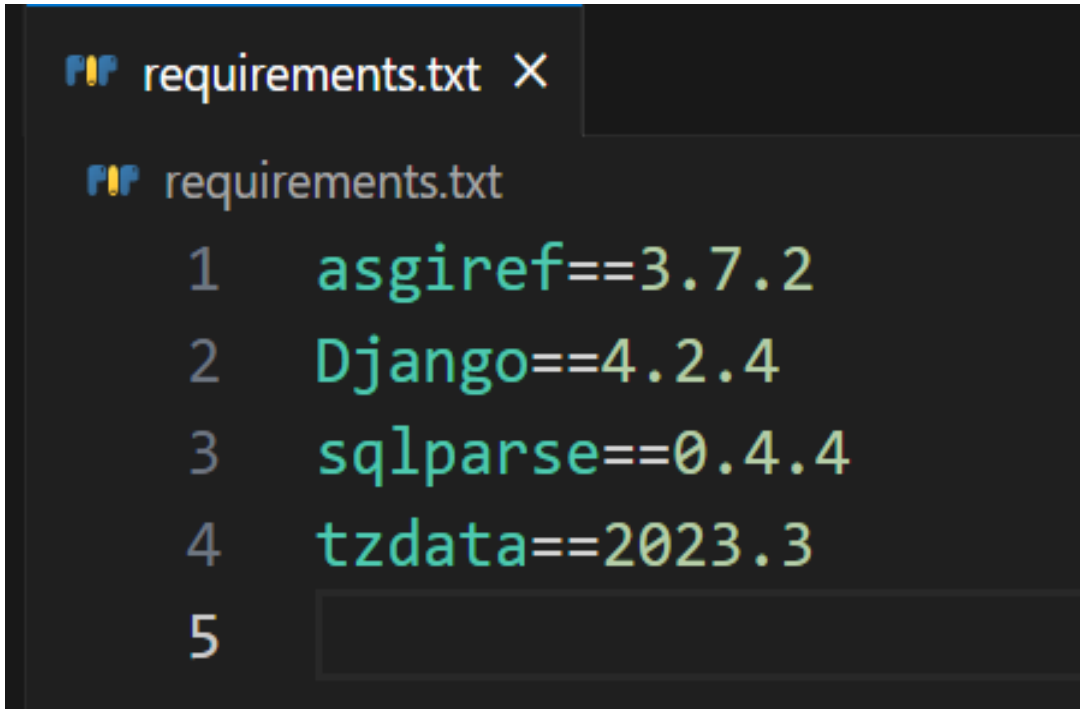
```
#créer et activer l'environnement virtuel
$ python -m venv my_env
$ my_env\Scripts\Activate
(my_env) $ python -m pip install django
```

Sauvegarder les dépendances

Étape par Étape

#sauvegarder les dépendances

(my_env) $ pip freeze > requirements.txt



Création projet et applications

Étape par Étape

```
# créer le projet django_project

(my_env) $ django-admin startproject meetings_planner   .

# lancer le projet dans VS code

(my_env) $ code .
```

```
# créer l'application website

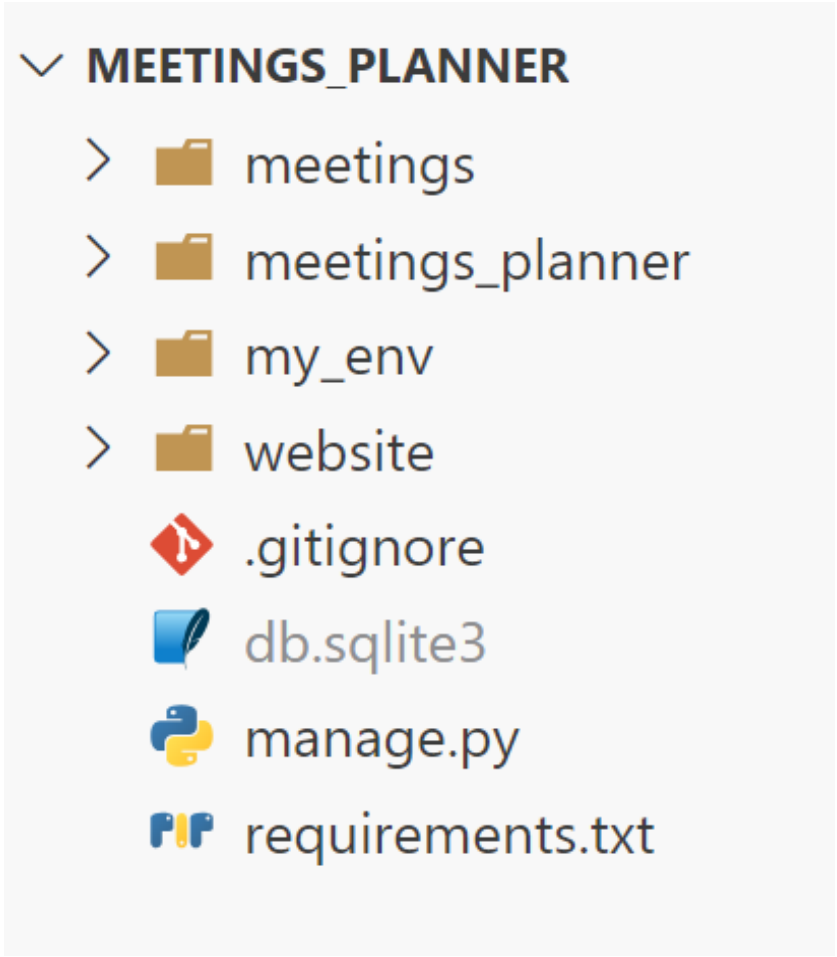
(my_env) $ python manage.py startapp website
```

```
# créer l'application meetings

(my_env) $ python manage.py startapp meetings
```

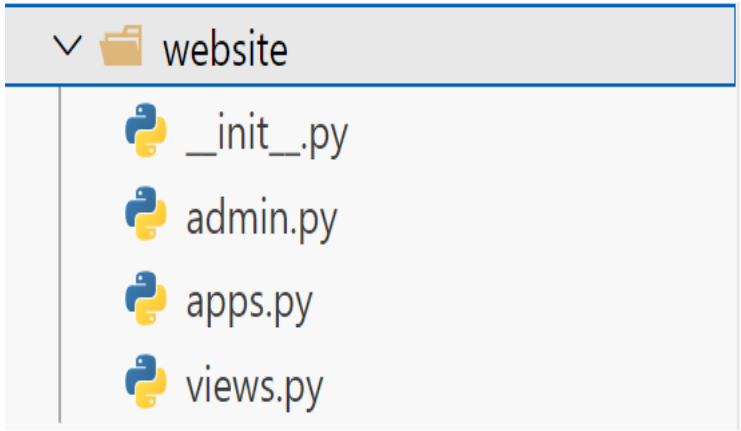
Structure du projet

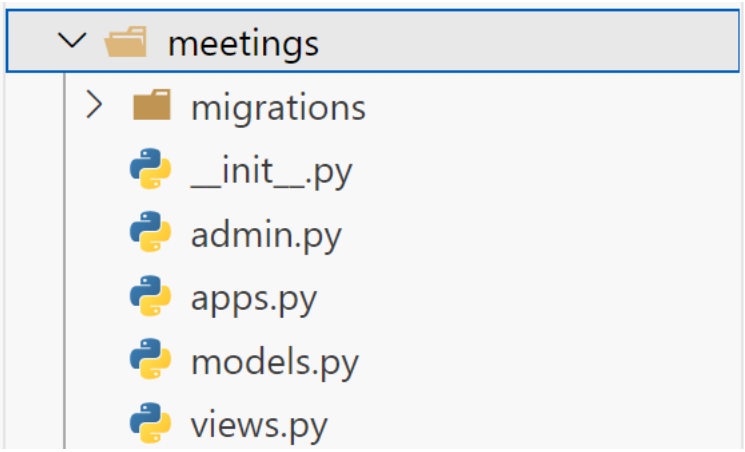
Étape par Étape



Structure des applications

Étape par Étape





Settings.py: Déclaration des applications

Étape par Étape

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'website',#new
    'meetings',#new
]
```

Application des migrations

Étape par Étape

```
# appliquer les migrations

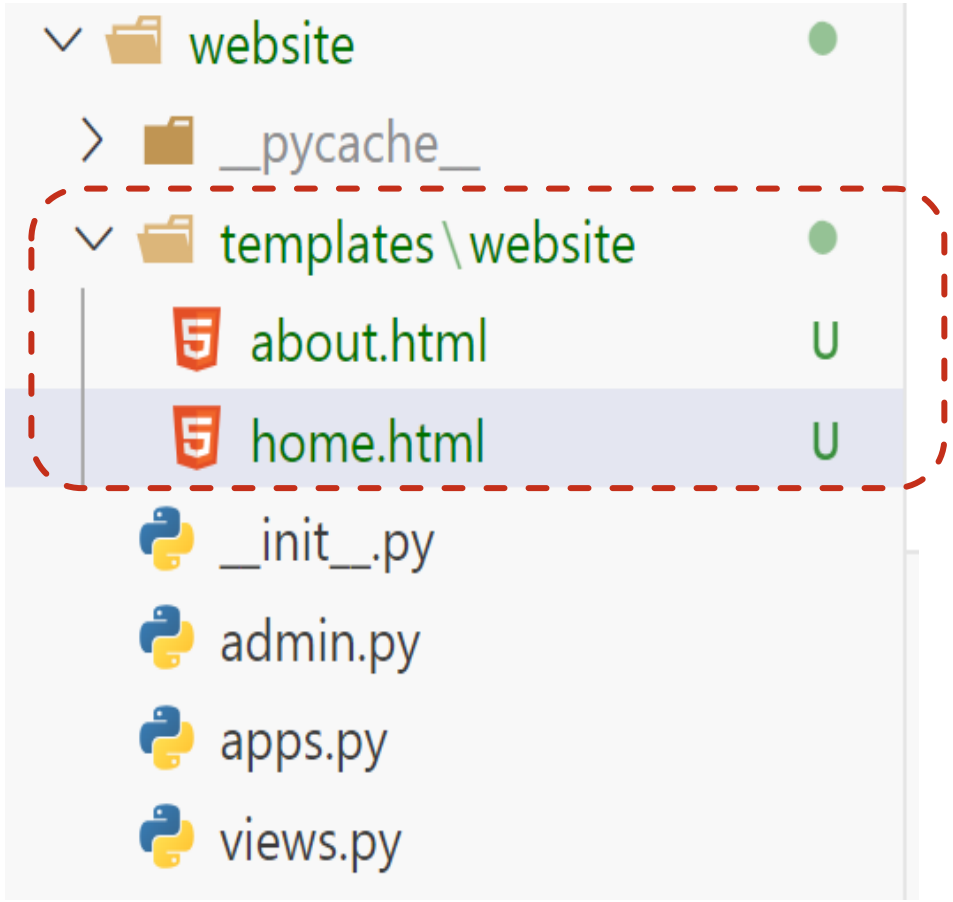
(my_env) $ python manage.py migrate
```

```
# démarrer le projet

(my_env) $ python manage.py runserver
```

website: home.html & about.html

Étape par Étape



website: home.html

Étape par Étape

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Meeting planer: home page</title>
</head>
<body>
    <h2>Meeting planner application from Tek-up</h2>
    <h3>Lite des réunions</h3>
</body>
</html>
```

website: about.html

Étape par Étape

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Meeting planer: about page</title>
</head>
<body>
    <h2>Meeting planner application from Tek-up</h2>

</body>
</html>
```

website: views.py

Étape par Étape

```
from django.shortcuts import render

# Create your views here.

def home_view(request):
    return render(request,"website/home.html")


def about_view(request):
    return render(request,"website/about.html")
```

website: urls.py

Étape par Étape

```
from django.urls import path

from . import views

#domain.com/website/...
urlpatterns=[
    path('',views.home_view, name='home'),
    path('about',views.about_view, name='about'),
]
```

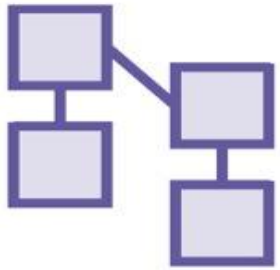
meetings_planner: urls.py

Étape par Étape

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('website/',include('website.urls')),#new
]
```

Créer le modèle



Django Models

Enregistrer les objets Python dans la BD

Les classes Models sont mappés à des tables

Les attributs sont mappés à des colonnes

SQL est généré

Create/Update tables (migrations)

Insert/Update/Delete lignes (admin)

Models

- Documentation: https://docs.djangoproject.com/fr/5.1/topics/db/models/

Démarche: Models

Étape par Étape

- Créer les classes modèles
- Créer les migrations
- appliquer les migrations
- Gérer le modèle avec Admin-interface

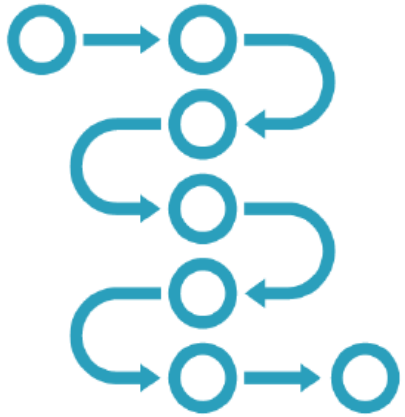
Migrations

Étape par Étape



Models

Classes Python
Associées aux tables de
la BD



Migrations

Ensemble de scripts
décrivant le schéma de
la BD

Show migrations

Étape par Étape

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
(.venv) PS C:\pc-nizar\Tek-up\AU2023-2024-S1\django\projects\meeting> python .\manage.py showmigrations
admin
 [ ] 0001_initial
 [ ] 0002_logentry_remove_auto_add
 [ ] 0003_logentry_add_action_flag_choices
auth
 [ ] 0001_initial
 [ ] 0002_alter_permission_name_max_length
 [ ] 0003_alter_user_email_max_length
 [ ] 0004_alter_user_username_opts
 [ ] 0005_alter_user_last_login_null
 [ ] 0006_require_contenttypes_0002
 [ ] 0007_alter_validators_add_error_messages
 [ ] 0008_alter_user_username_max_length
 [ ] 0009_alter_user_last_name_max_length
 [ ] 0010_alter_group_name_max_length
 [ ] 0011_update_proxy_permissions
 [ ] 0012_alter_user_first_name_max_length
contenttypes
 [ ] 0001_initial
 [ ] 0002_remove_content_type_name
sessions
 [ ] 0001_initial
website
```

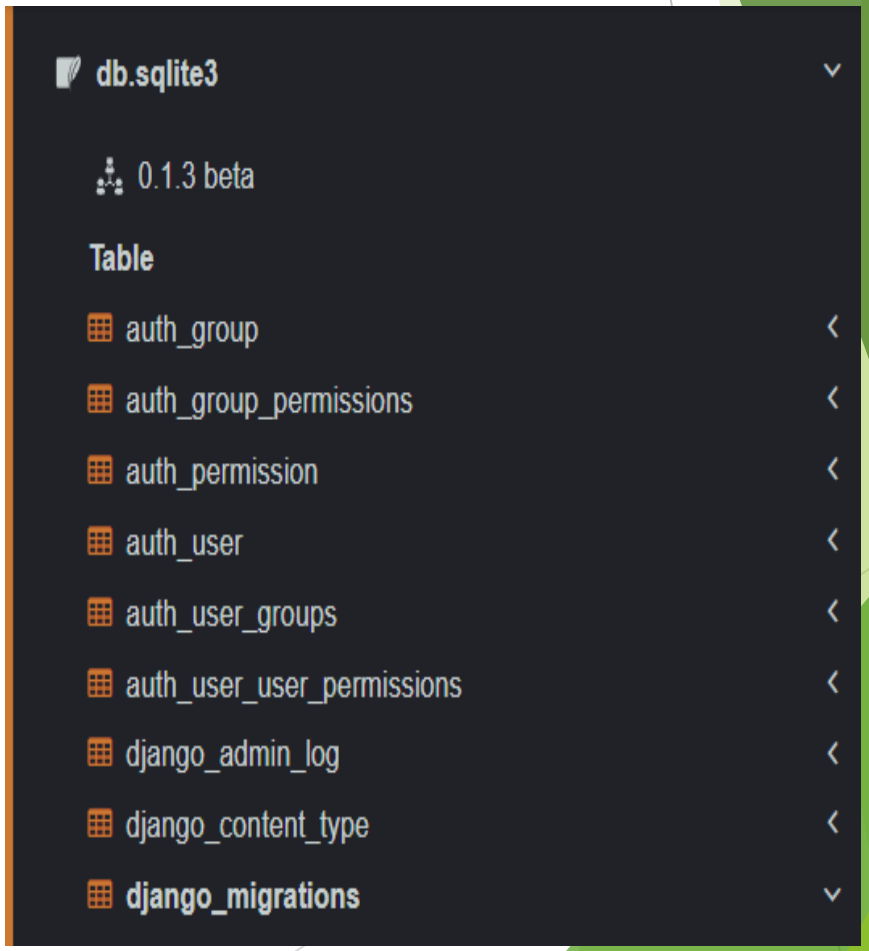
Appliquer les migrations

Étape par Étape

```
# appliquer les migrations

(my_env) $ python manage.py migrate
```

Afficher le schéma de la BD: https://sqliteonline.com/



Implémenter la classe modèle Meeting

Étape par Étape

meetings.models.py

```
from django.db import models

# Create your models here.

class Meeting(models.Model):
    title=models.CharField(max_length=200)
    date=models.DateField()
```

Créer et appliquer la migration

Étape par Étape

```
# créer la migration

(my_env) $ python manage.py makemigrations
```

```
# afficher SQL de la migration

(my_env) $ python manage.py sqlmigrate meetings 0001
```

```
# appliquer la migration

(my_env) $ python manage.py migrate meetings
```

Migration Workflow

```
# Step 1: Change Model code
# Step 2: Generate migration script (check it!)
python manage.py makemigrations
# Optional: Show migrations
python manage.py showmigrations
# Optional: Show SQL for specific migration
python manage.py sqlmigrate appname migrationname
# Step 3: Run migrations
python manage.py migrate
```

Django Admin

Deux parties de toute application web



End-users



Admin

django Admin

CRUD Model

Gestion Sécurité

Admin panel: éditer les modèles

Étape par Étape

- Enregistrer le modèle dans **admin.py**
- Créer un **superuser** pour pouvoir administrer l'application
- Démarrer l'application et connecter en tant que **superuser**
- Gérer le modèle

meetings.admin.py: enregistrer Meeting

Étape par Étape

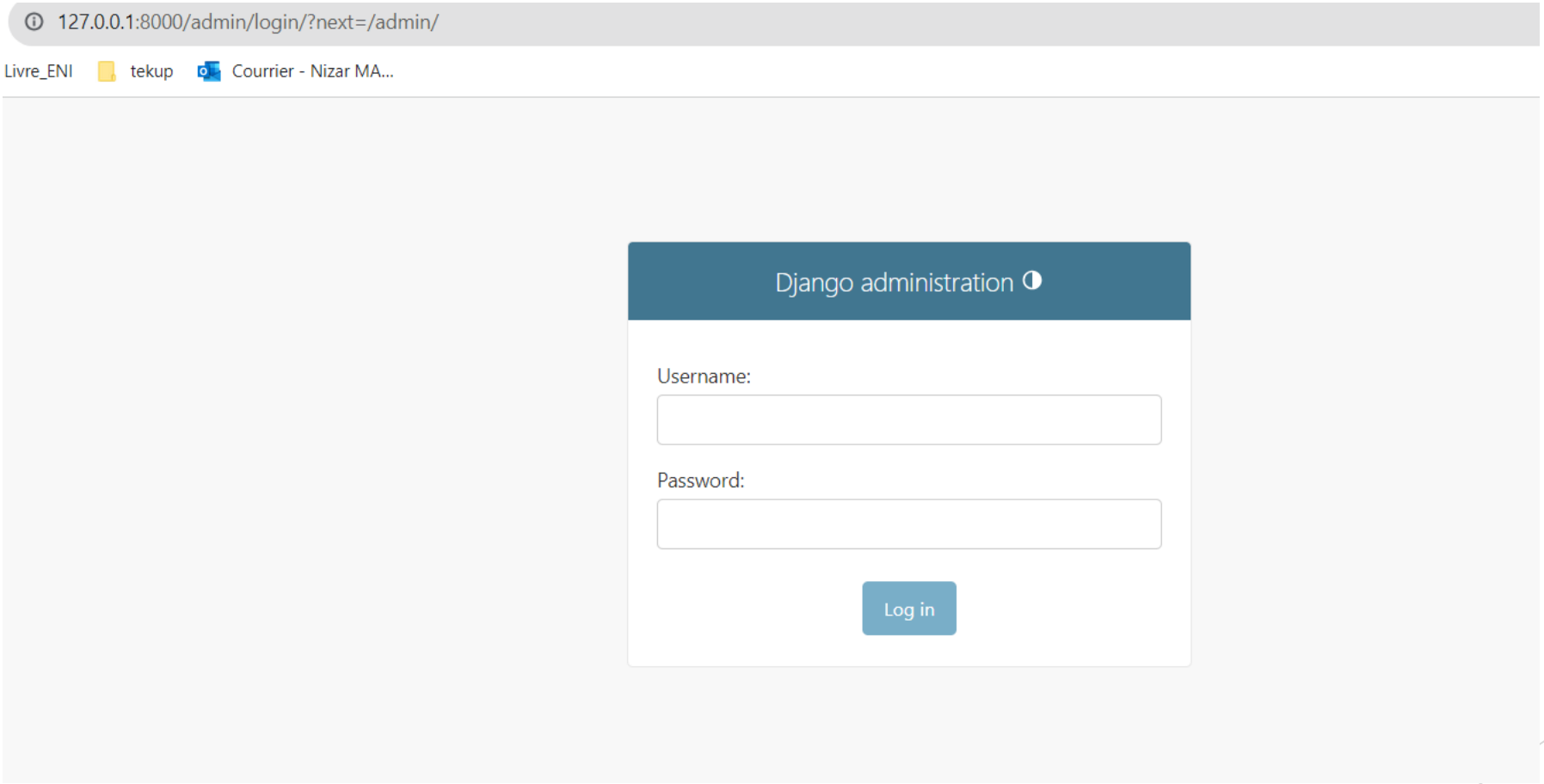
```
from django.contrib import admin

from .models import Meeting

admin.site.register(Meeting)
```

Admin interface

Étape par Étape



Admin interface: créer superuser

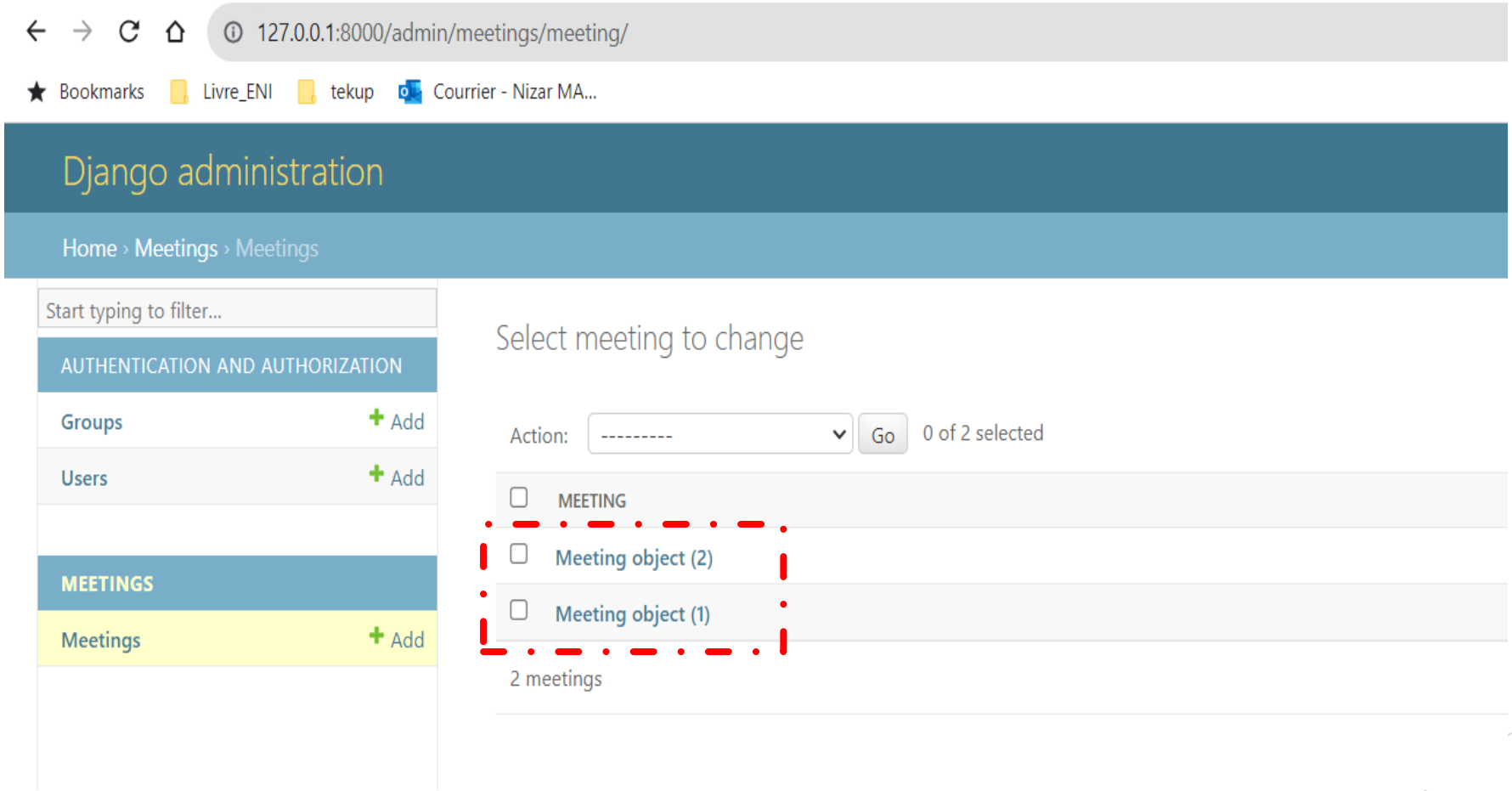
Étape par Étape

```
# créer superuser

(my_env) $ python manage.py createsuperuser
```

Admin interface: ajouter des meetings

Étape par Étape



Exercice: Mettre à jour Meeting

Étape par Étape

```
from django.db import models
from datetime import time

# Create your models here.

class Meeting(models.Model):
    title=models.CharField(max_length=200)
    date=models.DateField()
    start_time=models.TimeField(default=time(9))
    duration=models.IntegerField(default=1)

    def __str__(self):
        return f"{self.title} at {self.start_time} on {self.date}"
```

Créer et appliquer la migration

Étape par Étape

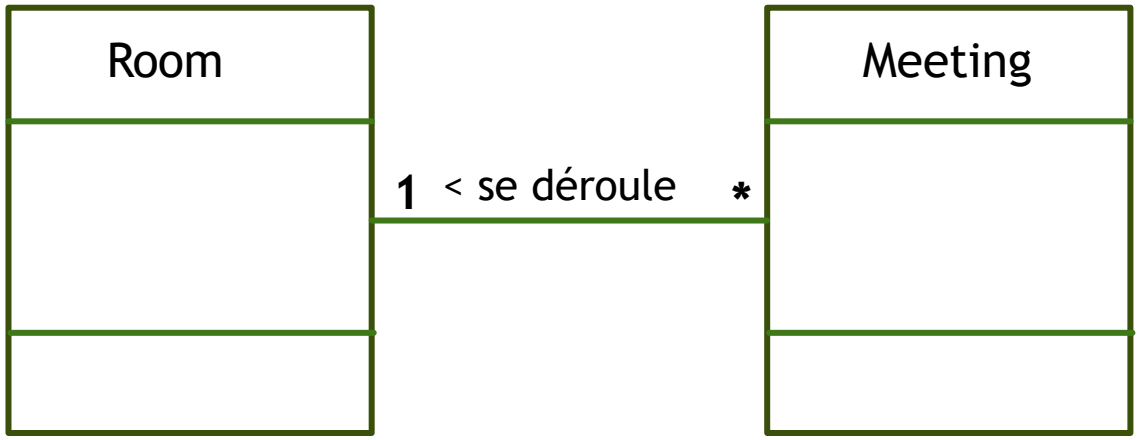
```
# créer les migrations

(my_env) $ python manage.py makemigrations
```

```
# appliquer les migrations

(my_env) $ python manage.py migrate meetings
```

Model Relations: One-To-Many



meetings.models.py: Room

```
class Room(models.Model):
    name = models.CharField(max_length=50)
    floor = models.IntegerField()
    room_number = models.IntegerField()

    def __str__(self):
        return f"{self.name}: room {self.room_number} on floor {self.floor}"
```

Ajouter association one to many

Étape par Étape

```
class Meeting(models.Model):
    title = models.CharField(max_length=200)
    date = models.DateField()
    start_time = models.TimeField(default=time(9))
    duration = models.IntegerField(default=1)
    room = models.ForeignKey(Room, on_delete=models.CASCADE)

    def __str__(self):
        return f"{self.title} at {self.start_time} on {self.date}"
```

Créer et appliquer la migration

Étape par Étape

```
# créer les migrations

(my_env) $ python manage.py makemigrations
```

```
# appliquer les migrations

(my_env) $ python manage.py migrate meetings
```

Templates

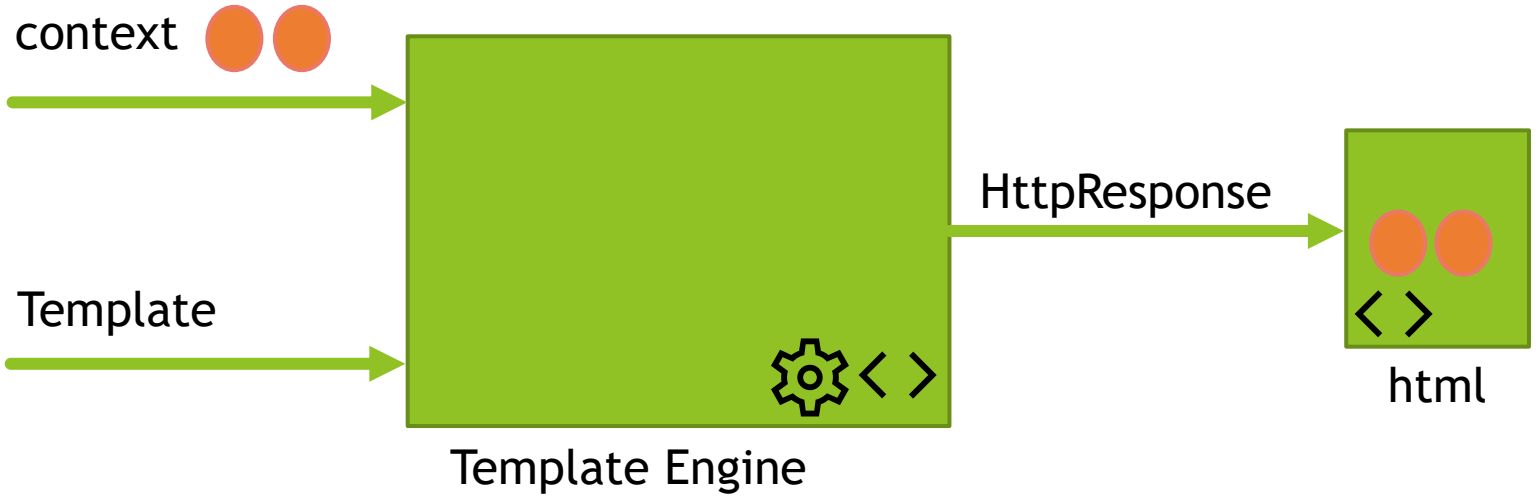
Générer les pages web

Template variables

Template tags

Template inheritance

Templates:



Templates:

HTML

+

Django Template Langage:

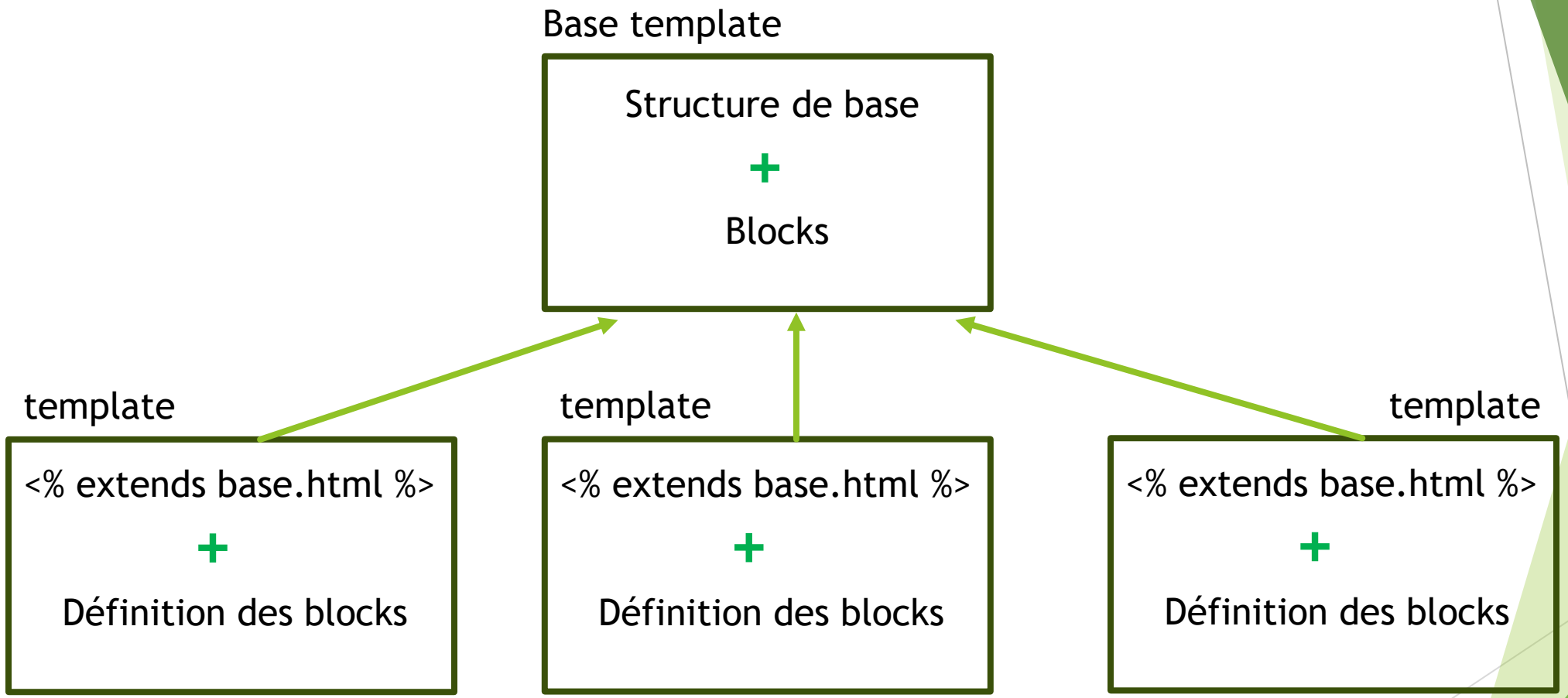
Template variables: {{ context variable}}

Template Tags: <% for var in context vars %>

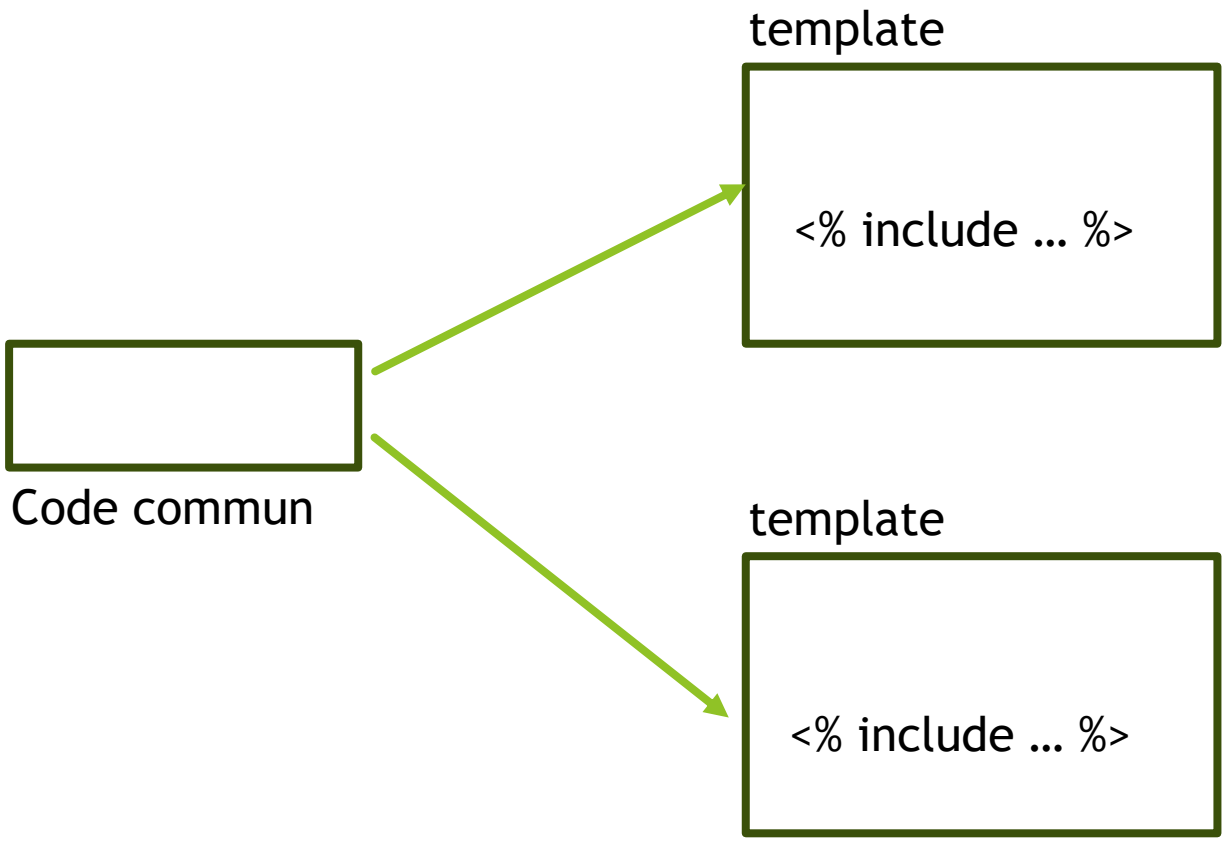
Template filters: {{ variable | filtername }}

Template

Templates: héritage



Template: include



website.views

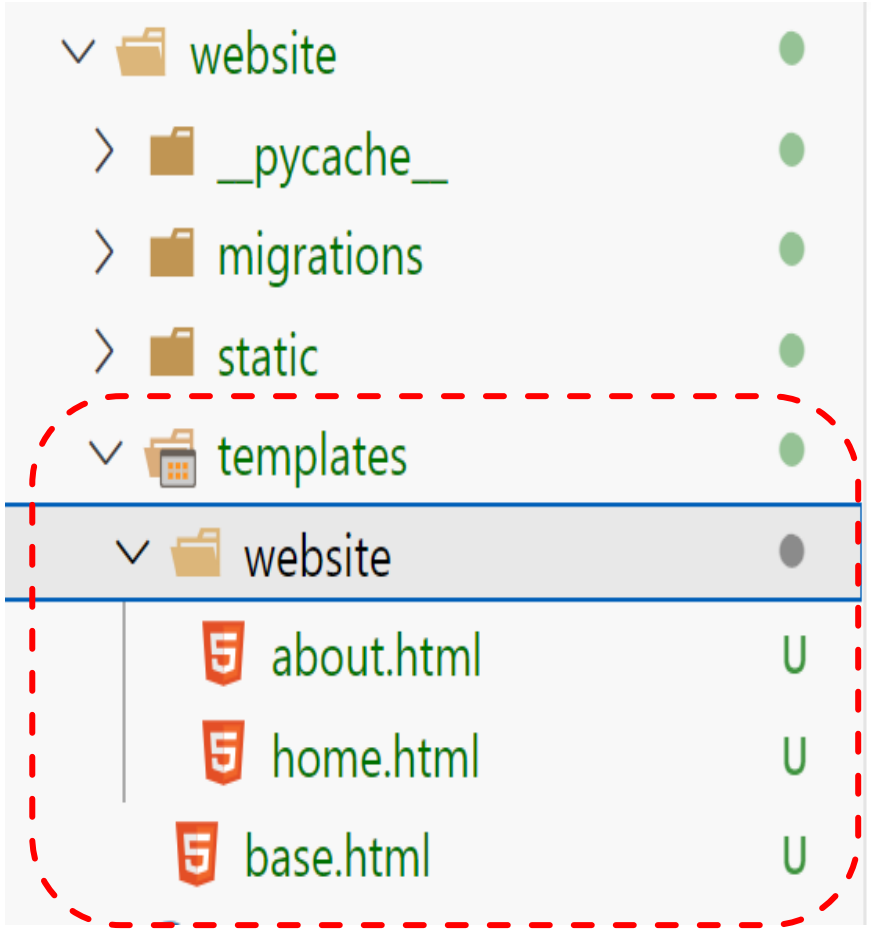
Étape par Étape

```
def home_view(request):
    context={'nbre_meeting': Meeting.objects.count()}
    return render(request, "website/home.html",context=context)

def about_view(request):
    return render(request,"website/about.html")
```

Templates: héritage

Étape par Étape



Templates: héritage (base.html)

Étape par Étape

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

    <title>{% block title %}{% endblock %}</title>

    </head>
<body>
    {% block content %}
    {% endblock %}
</body>
</html>
```

Templates: héritage (about.html)

Étape par Étape

```
{% extends "base.html" %}

{% block title %}about{% endblock %}

{% block content %}
    <h2>Meeting planner application from Tek-up</h2>

{% endblock %}
```

website: home.html v1

Étape par Étape

```
{% extends "base.html" %}

{% block title %}HOME{% endblock %}

{% block content %}
    <h2>Welcome to the meeting planner</h2>
    <p>there are currently {{nbre_meeting}} meetings</p>

{% endblock %}
```

meetings.views

Étape par Étape

```
def detail(request, id):
    meeting = Meeting.objects.get(pk=id)
    return render(request, "meetings/detail.html", {"meeting": meeting})
```

```
def detail(request, id):
    meeting = get_object_or_404(Meeting, id)
    return render(request, "meetings/detail.html", {"meeting": meeting})
```

website: home.html v2

Étape par Étape

```
<h2>Meetings</h2>
    <ul>
        {% for meeting in meetings %}
            <li>
                <a href="{% url 'detail' meeting.id %}">
                    {{ meeting.title }}
                </a>
            </li>
        {% endfor %}
    </ul>
```

website.views v3

Étape par Étape

```
def home_view(request):
    context={'meetings': Meeting.objects.all()}
    return render(request, "website/home.html",context=context)
```

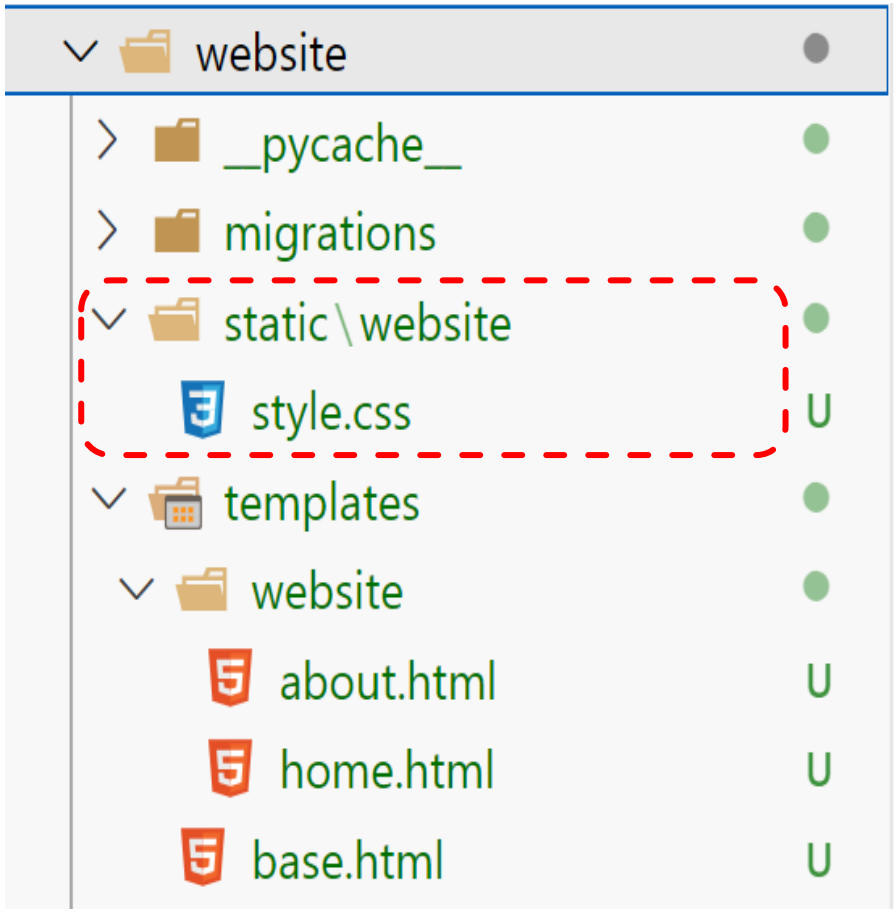
website: home.html v3

Étape par Étape

```
<h2>Meetings</h2>
    <ul>
        {% for meeting in meetings %}
            <li>
                <a href="{% url 'detail' meeting.id %}">
                    {{ meeting.title }}
                </a>
            </li>
        {% endfor %}
    </ul>
    <a href="{% url 'rooms' %}">Rooms list</a>
```

Templates: static files

Étape par Étape




```
body {
    font-family: sans-serif;
    color: cornflowerblue;
    background-color: floralwhite;
}
```

Templates: static files (settings.py)

Étape par Étape

```
# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/4.2/howto/static-files/

STATIC_URL = 'static/'
```

Templates: static files (base.html)

Étape par Étape

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}{% endblock %}</title>
    <link rel="stylesheet"
          href="{% static 'website/style.css' %}">
</head>
<body>
    {% block content %}
    {% endblock %}
</body>
</html>
```

Exercice

- Ajouter une page qui liste tous les rooms:
  - Views
  - template
  - url mapping

django forms

Valider les entrées utilisateurs

Automatise le code répétitif

sécurisé

Hautement personnalisé

Formulaire: Meeting

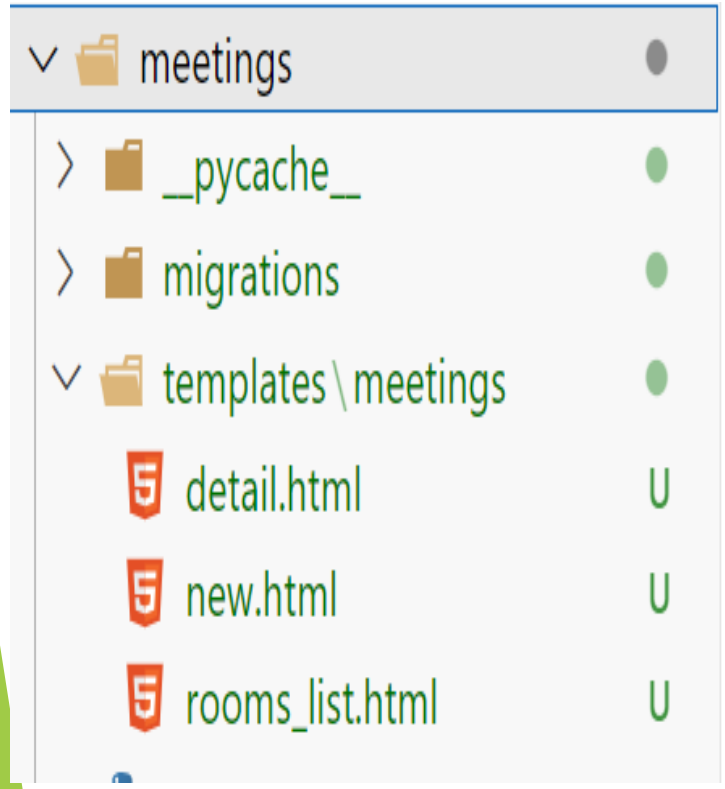
Étape par Étape

```
class MeetingForm(ModelForm):
    class Meta:
        model = Meeting
        fields = '__all__'
        widgets = {
            'date': DateInput(attrs={"type": "date"}),
            'start': TimeInput(attrs={"type": "time"}),
            'duration': TextInput(attrs={"type": "number","min":"1", "max":"4"})
        }

    def clean_date(self):
        d = self.cleaned_data.get("date")
        if d < date.today():
            raise ValidationError("Meetings cannot be in the past")
        return d
```

Formulaire: new.html

Étape par Étape




```
{% extends "base.html" %}
{% block title %}New Meeting{% endblock %}

{% block content %}
<h1>Plan a new meeting</h1>
<form method="post">
    <table>
        {{ form }}
    </table>
    {% csrf_token %}
    <button type="submit">Create</button>
</form>
{% endblock %}
```

Exercice

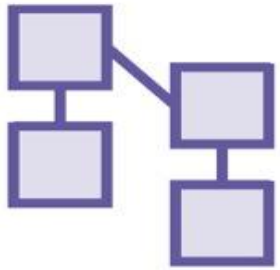
- Ajouter tout le nécessaire pour permettre l'ajout d'une salle de réunion



- Architecture MVT
- Models: définition, manipulation, associations
- Admin-panel: CRUD, sécurité
- Templates: variables, tags, static files, héritage
- Forms: mapping entre templates et Models

TEK-UP University

django Models



Résultats

À la fin de cette séance, vous serez en mesure de:

- Maitriser le composant **Models** dans l'architecture **MTV** de **django**
  - django ORM
  - django Models
  - Manipulation des modèles
  - Migrations, optimisation

Éléments du contenue

- Modéliser vos données
- Écrire des requêtes avancées
- Personnaliser le comportement de vos Models
- Créer et appliquer des migrations
- Optimiser les performances

Model-Template-View

Model	Template	View
Données	Présentation	Comportement
Classes Modèle Mapped to DB	Générer HTML	Fonction python Mapped to URL
Model	View	Controller

ORM

Object-Relational Mapping

Un autre nom de la couche **Models**

Relational data

- Lignes dans des tables de base de données
- SQL

Python data

- Classes et objets, valeurs et variables

ORM bridges the gaps

- les classes modèles sont mappés à des tables
- SQL est généré
- coder uniquement en python

django Models

Mapped to DB tables

Générer UI (ModelForm)

Valider Forms

Générer Admin interface

Ajouter custom methods

django Models

Bases de données supportées:

- PostgreSQL, MariaDB, MySQL, Oracle, SQLite
- avec packages: DB2, MS SQL, ...

Projet de démonstrations

- Application web de e-commerce (gestion des produits)
- Focaliser le développement dans la couche Model
- Utiliser deux BD:
  - SQLite
  - PostgreSQL

Création du projet

Étape par Étape

```
#créer un dossier pour le projet
$ cd framework_django\projects\django_models
$ mkdir e_commerce
$ cd e_commerce
```

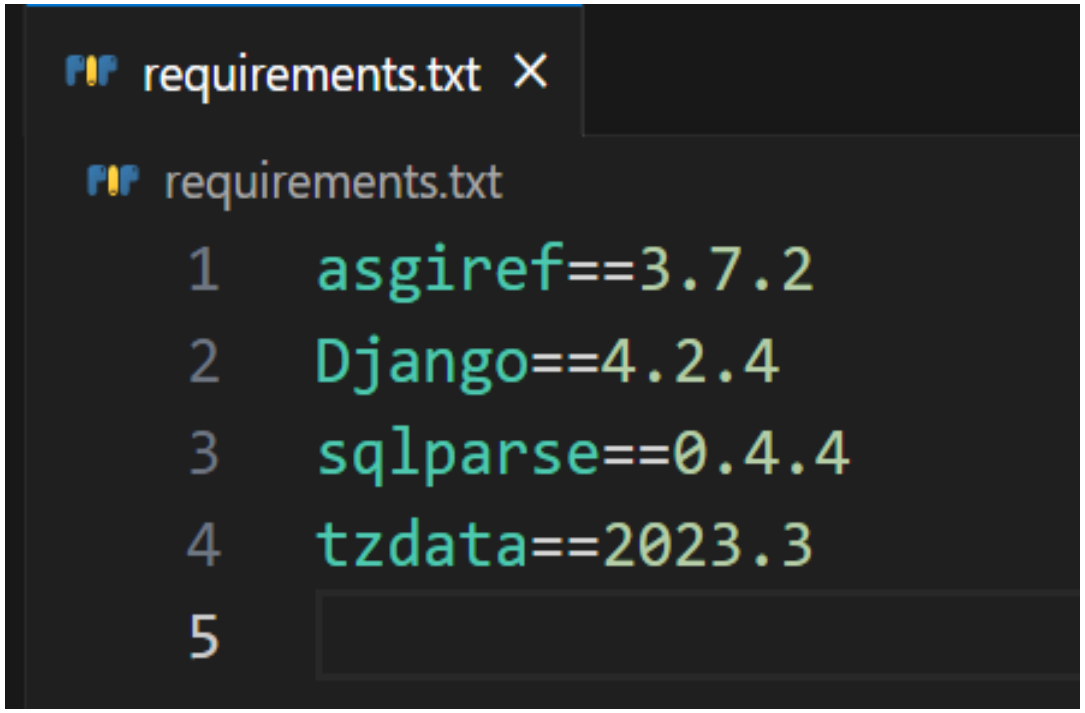
```
#créer et activer l'environnement virtuel
$ python –m venv my_env
$ my_env\Scripts\Activate
(my_env) $ python -m pip install django
```

Sauvegarder les dépendances

Étape par Étape

#sauvegarder les dépendances

(my_env) $ pip freeze > requirements.txt



Création du projet et applications

Étape par Étape

```
# créer le projet e_commerce_site
(my_env) $ django-admin startproject  e_commerce_site  .
# lancer le projet dans VS code
(my_env) $ code .
```

```
# créer l'application products
(my_env) $ python manage.py startapp products
```

settings.py: installer l'application

Étape par Étape

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'products.apps.ProductsConfig',#new
]
```

Installation de PostgreSQL

- https://www.postgresql.org/download/



settings.py: Configuration du Log

Étape par Étape

```
#logging all SQL
LOGGING={
    "version": 1,
    'handlers': {
        'console': {
            'level': 'DEBUG',
            'class': 'logging.StreamHandler',
        }
    },
    'loggers': {
        'django.db.backends': {
            'level': 'DEBUG',
            'handlers': ['console'],
        }
    }
}
```

settings.py: configuration du

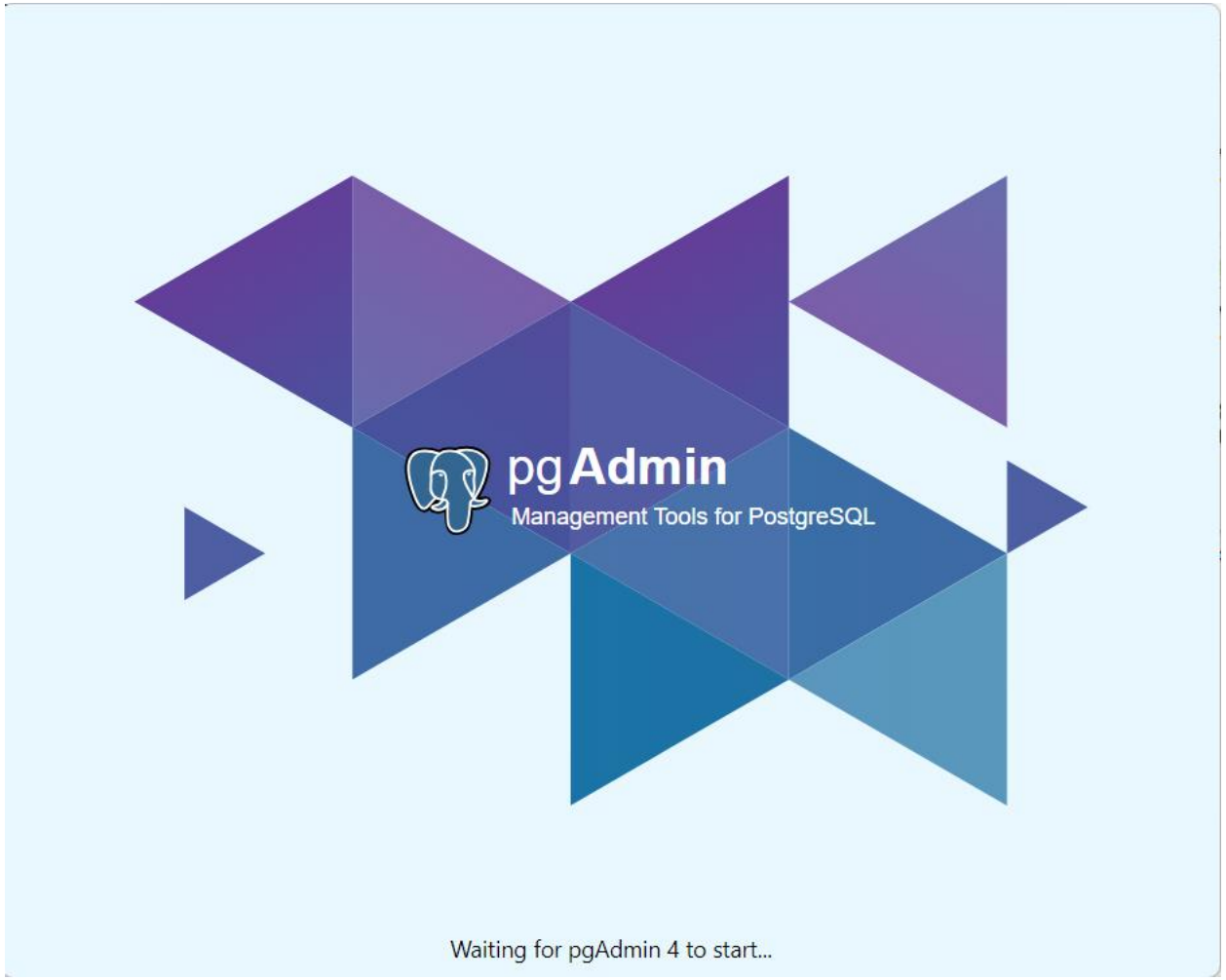


Étape par Étape

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'products_db',
        'HOST': '127.0.0.1',
        'USER': 'postgres',
        'PASSWORD': 'adminadmin',
    }
}
```

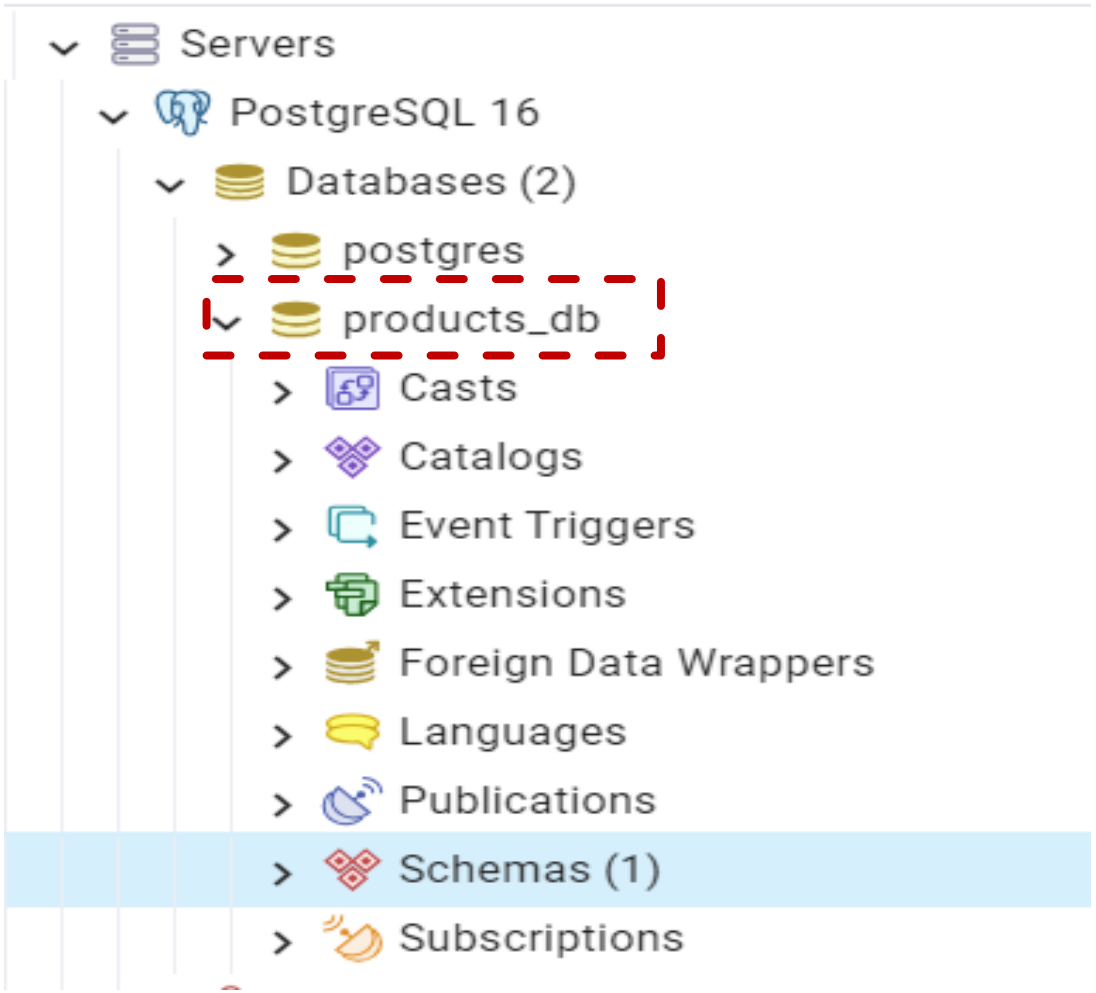
PostgreSQL: créer la base de données

Étape par Étape



PostgreSQL: créer la base de données

Étape par Étape



Installer psycopg2

Étape par Étape

```
python –m pip install psycopg2

pip freeze > requirements.txt
```

django Models: classe et instance

Démarche

Étape par Étape

- Créer classe Modèle
- Créer et appliquer les migrations
- Manipulation des instances du modèle:
  - Create
  - Update
  - Delete
- Différents backend

products.models.py: modèle Product

Étape par Étape

```
from django.db import models

class Product(models.Model):
    name=models.CharField(max_length=100)
    stock_count=models.IntegerField(default=0)
    price=models.DecimalField(max_digits=6,decimal_places=2)
```